

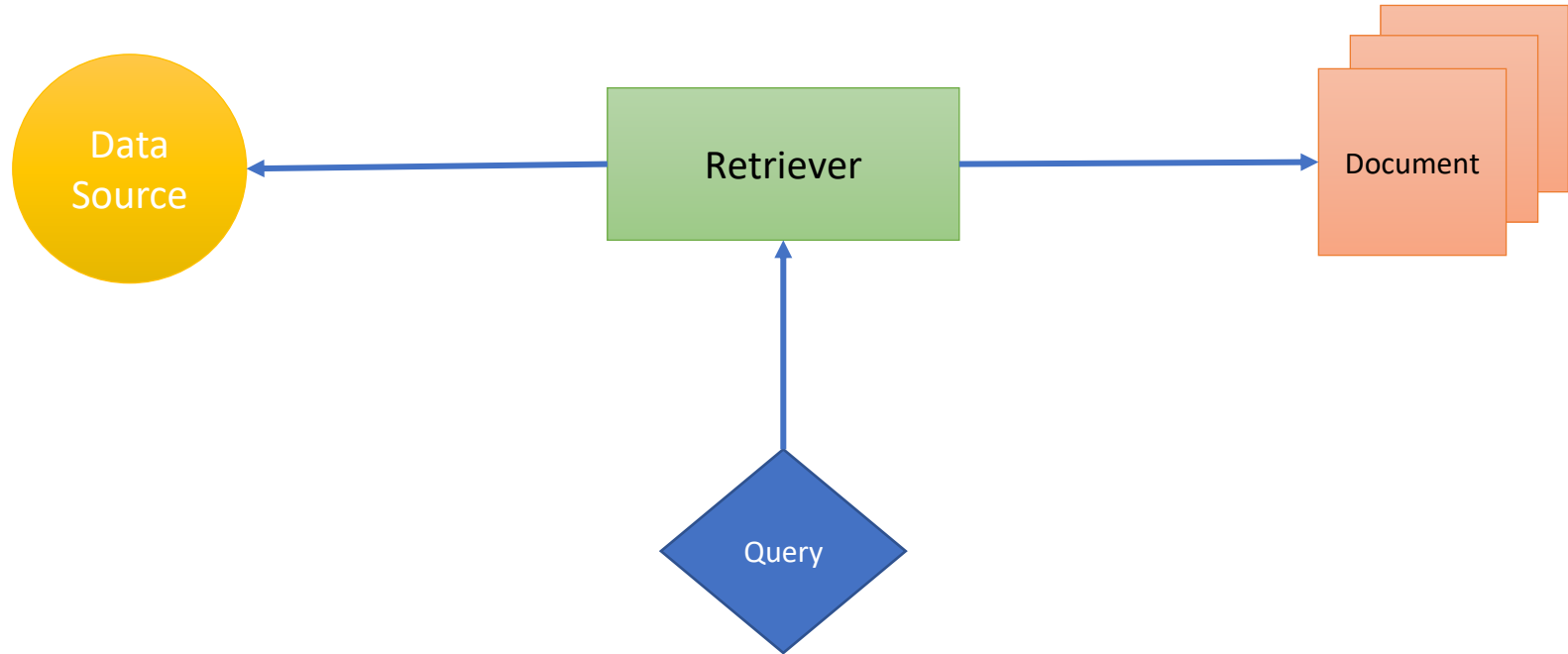
Retrievers in LangChain

What are Retrievers?

A retriever is a component in LangChain that fetches relevant documents from a data source in response to a user's query.

Key Points:

- There are multiple types of retrievers
- All retrievers in LangChain are runnables



Types of Retrievers

Wikipedia Retriever

Queries Wikipedia API to fetch relevant content

Vector Store Retriever

Searches vector stores using semantic similarity

ArxivRetriever Retriever

arxiv_retriever is a lightweight command-line tool designed to automate the retrieval of computer science papers from ArXiv.

Maximal Marginal Relevance

Reduces redundancy while maintaining relevance

Multi-Query Retriever

Generates multiple queries from a single query

Contextual Compression

Compresses documents to keep only relevant parts

Additional Retrievers

BM25, ParentDocument, TimeWeighted, SelfQuery, etc.

Wikipedia Retriever

A Wikipedia Retriever is a retriever that queries the Wikipedia API to fetch relevant content for a given query.

How It Works:

1. Give it a query (e.g., 'Albert Einstein')
2. It sends the query to Wikipedia's API
3. It retrieves the most relevant articles
4. Returns them as LangChain Document objects

Vector Store Retriever

A Vector Store Retriever in LangChain is the most common type of retriever that lets you search and fetch documents from a vector store based on semantic similarity using vector embeddings.

How It Works:

1. Documents stored in vector store (FAISS, Chroma, Weaviate)
2. Each document converted to dense vector using embedding model
3. User query entered:
 - Turned into a vector
 - Compared with stored vectors
 - Top-k most similar documents retrieved

Maximal Marginal Relevance (MMR)

MMR is an information retrieval algorithm designed to reduce redundancy in the retrieved results while maintaining high relevance to the query.

Problem It Solves:

Regular search returns documents that are: very similar to each other, repeating same info, lacking diverse perspectives

MMR Solution:

1. Pick the most relevant document first
2. Pick next most relevant AND least similar to already selected
3. Balance between relevance and diversity

Multi-Query Retriever

Single query may not capture all ways information is phrased. Uses LLM to generate multiple semantically different query versions.

How It Works (4 Steps):

1. Takes your original query
2. Uses LLM (GPT-3.5) to generate multiple versions
3. Performs retrieval for each sub-query
4. Combines and deduplicates results

Example :

Query :

How can I stay Healthy?

Could mean:

What should I eat?

How often should I exercise?

How can I manage stress>

How can I stay Healthy?

what are the best foods to maintain good health?

how often should I exercise to stay fit?

what lifestyle habits improve mental and physical wellness?

How can I boost my immune system naturally?

Contextual Compression Retriever

Improves retrieval quality by compressing documents after retrieval - keeps only relevant content based on query.

Problem Without Compression:

Returns entire paragraph when only 1 sentence is relevant. Irrelevant noise wastes context window and confuses LLM.

How It Works:

1. Base Retriever retrieves N documents
2. Compressor (LLM) applied to each document
3. Keeps only relevant parts to query
4. Irrelevant content discarded

Additional Retriever Types

BM25Retriever Probabilistic ranking for keyword-based retrieval

ParentDocumentRetriever Retrieves parent documents for context

TimeWeightedVectorRetriever Combines similarity with recency scoring

SelfQueryRetriever LLM-powered with self-generated filters

MultiVectorRetriever Multiple vector representations

EnsembleRetriever Combines results from multiple retrievers

ArxivRetriever Searches arXiv research papers

Key Takeaways

1. Retrievers fetch relevant documents from data sources based on queries
2. Vector Store Retrievers use semantic similarity - most common approach
3. MMR balances relevance and diversity, reducing redundancy
4. Multi-Query generates variations to overcome terminology gaps
5. Contextual Compression optimizes context with relevant content only
6. LangChain supports diverse retrievers for different use cases